TITLE

Design and Implementation of Dynamic Approval Workflows using Finite State Machines and Context-Aware Access Control

Quang Phuoc HUA

VNU-HCM High School for the Gifted

ABSTRACT

In the era of digital transformation, traditional administrative approval processes often create significant operational bottlenecks, particularly in geographically distributed enterprises. This research presents the design and implementation of an E-submission System aimed at resolving remote approval challenges for a multi-branch company in Vietnam.

Unlike rigid off-the-shelf software solutions, this system utilizes a hybrid low-code architecture. The core contribution of this study is the application of *Finite State Machines* (FSM) to model complex, non-linear approval workflows. Furthermore, the system integrates custom scripting to handle *thread synchronization*—specifically solving the "Wait-for-Join" problem in parallel advisory processes where multiple stakeholders must provide input before a transition occurs. Additionally, a *Dynamic Attribute-Based Access Control* (ABAC) mechanism was implemented to automatically assign permissions based on user input data in real-time.

The system has been deployed in a real-world environment, resulting in a streamlined workflow that eliminates manual routing errors, ensures process integrity, and significantly reduces approval turnaround time.

Keywords: Finite State Machine, Workflow Automation, Rapid Application Development, Synchronization Logic, Context-Aware Access Control.

1. INTRODUCTION

1.1. Motivation and Problem Statement

The rapid shift towards remote work has exposed critical inefficiencies in traditional paper-based administrative procedures. A specific case study is observed at a logistics and trading company in Vietnam, where the Board of Directors and the operational teams are located in different cities (Hanoi and Ho Chi Minh City).

The physical distance created a significant "bottleneck" in the approval of internal proposals (*Tờ trình*). Physical documents often took days to travel between branches, and email-based approvals lacked state tracking, leading to lost requests and a lack of accountability. While many "No-code" tools exist in the market, they often fail to handle complex, conditional logic required by the enterprise, such as distinguishing between CEO-level and Board-level approvals or enforcing strict synchronization rules for advisory groups.

1.2. Technical Challenges

The development of this system addressed three primary technical challenges. First, Complex Branching Logic was required to handle various proposal types, necessitating routing paths modeled as Directed Acyclic Graphs to dynamically determine the subsequent state. Second, Parallel Synchronization posed a difficulty in the advisory process due to the dynamic number of advisors; this required a solution beyond standard linear workflows to enforce a specific condition—where the set of responses must satisfy the required subset—before the process could proceed. Lastly, Dynamic Security was critical to avoid hard-coded permissions, demanding a Context-Aware Security model where access rights are granted dynamically at runtime based on the specific department selected by the user.

1.3. Approach and Contribution

To address these challenges, this project implements a hybrid approach. I utilized a Low-code platform for the User Interface (UI) to ensure rapid deployment. However, for the core business logic, I implemented custom scripts (Imperative Logic) to handle state transitions and synchronization.

This paper documents the system architecture, the design of the *Finite State Machine* used for workflow modeling, and the specific algorithms used for dynamic role binding and parallel synchronization.

2. THEORETICAL BACKGROUND

To provide a rigorous foundation for the implemented solution, this research draws upon three core computer science concepts: Finite State Machines for process modeling, Workflow Patterns for concurrency control, and Attribute-Based Access Control for dynamic security.

2.1. Finite State Machines (FSM) in Process Modeling

A Finite State Machine is a mathematical model of computation used to design both computer programs and sequential logic circuits. In the context of this E-submission system, the lifecycle of a proposal is modeled as a deterministic finite automaton (DFA).

Formally, the workflow is defined as a 5-tuple $M = (Q, \Sigma, \delta, q0, F)$, where Q represents the finite set of operational states (e.g., Draft, Pending Review, Approved), and Σ denotes the finite set of input events triggering transitions (e.g., Submit, Approve, Reject). The core logic is governed by the transition function δ, which maps a specific state and event to a subsequent state ($Q \times \Sigma \rightarrow Q$). Finally, $q0$ designates the initial state (Draft), while F represents the set of final states where the process concludes (Finalized, Archived).

By applying FSM theory, the system ensures that a document cannot exist in an undefined state and can only transition to a valid next state upon a specific trigger, thereby eliminating procedural ambiguity.

2.2. Workflow Synchronization Patterns

A critical challenge in digitizing administrative processes is handling concurrency. This research specifically addresses the "Synchronization" pattern (often referred to as the AND-Join pattern in workflow theory).

In a parallel routing scenario, a single thread of control splits into multiple branches (Split) and must eventually merge back into a single thread (Join). The system must enforce a blocking mechanism where the subsequent activity is triggered only when all parallel branches have been completed. This concept underpins the custom scripting logic developed for the advisory process described in Section 3.

2.3. Attribute-Based Access Control (ABAC)

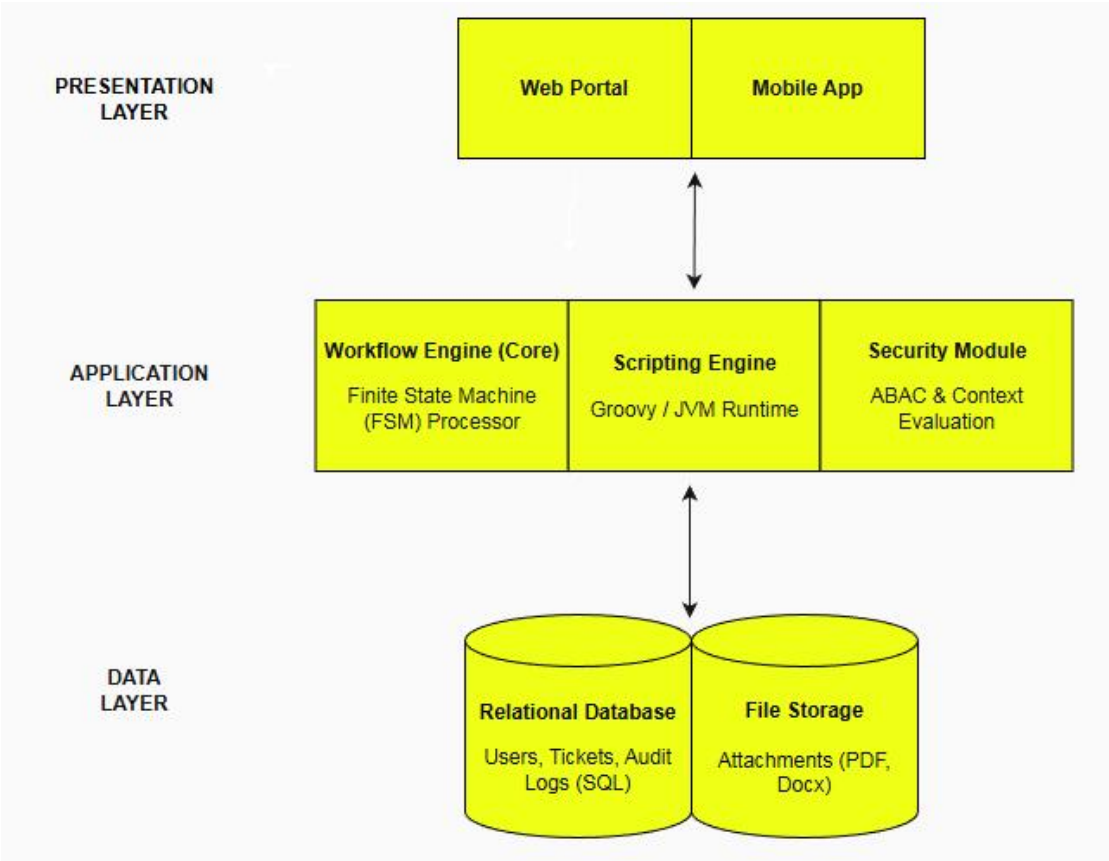
Traditional systems often use Role-Based Access Control (RBAC), where permissions are assigned to static roles. However, complex enterprises require a more granular approach. This system implements Attribute-Based Access Control (ABAC), where access decisions are based on the attributes of the subject (user), the object (proposal), and the environment conditions.

In this implementation, the "Department" field selected by the user acts as a dynamic attribute. The access control policy is evaluated at runtime: *Allow access IF User.Department matches Ticket.Target_Department*. This ensures the Principle of Least Privilege, granting access only to relevant personnel for a specific transaction without permanently elevating their privileges.

3. SYSTEM DESIGN AND IMPLEMENTATION

3.1. High-Level System Architecture

The system follows a classic **Three-Tier Architecture**, ensuring a clean separation of concerns between the user interface, business logic, and data storage.



Presentation Layer: Built using the Low-code platform's reactive UI framework, accessible via web browsers and mobile devices.

Application Layer (The Core): This layer hosts the Workflow Engine (managing FSM states), the Scripting Engine (executing Groovy logic for synchronization), and the Security Module (enforcing ABAC policies).

Data Layer: Utilizes a relational database (SQL) for structured data (Users, Tickets) and an object storage service for unstructured documents.

3.2. System Architecture and Workflow Modeling

The system is built upon a Low-code development platform to accelerate User Interface (UI) generation. However, the core logic is orchestrated using a Hybrid Architecture that combines declarative configuration with imperative scripting.

The approval workflows are modeled as Finite State Machines (FSM). This ensures that a proposal always exists in a deterministic state and transitions only occur when specific conditions are met.

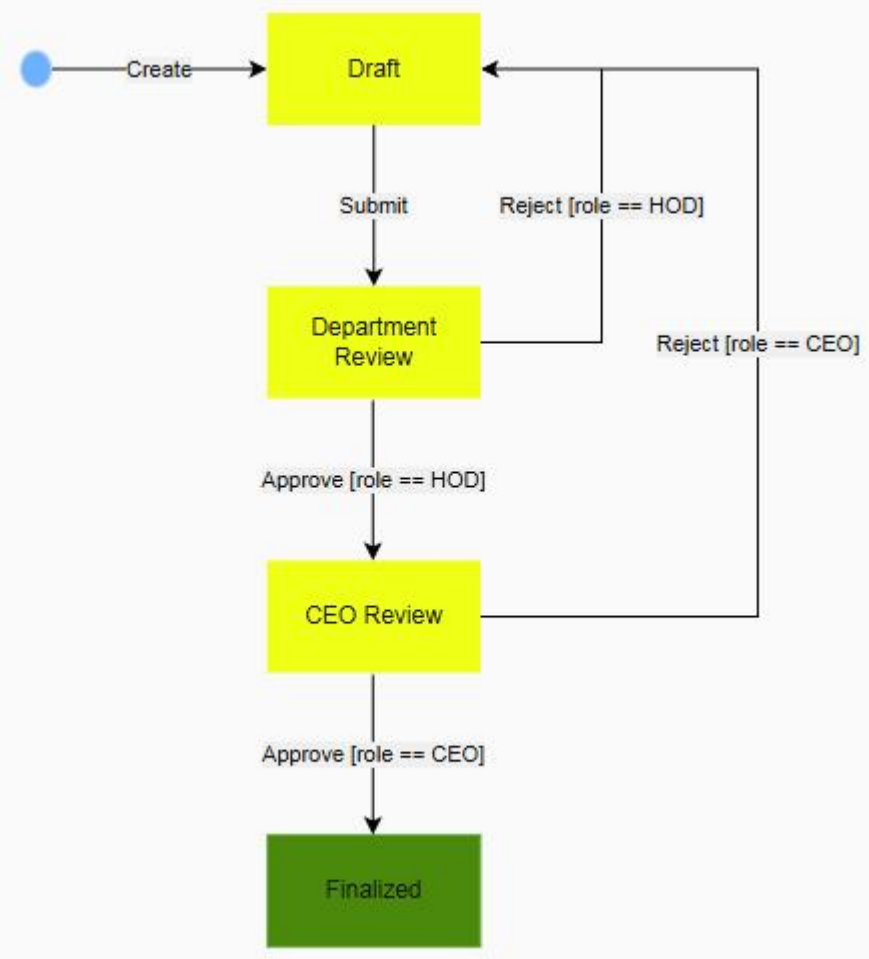


Figure 1: State Transition Diagram of the proposal approval workflow modeled as a Finite State Machine.

As illustrated in *Figure 1*, the workflow directs the proposal through sequential states (e.g., Draft → Department Head → CEO → Finalized). For these linear transitions, the system utilizes the platform's native configuration engine, where transitions are guarded by static role checks (e.g., Current_User must possess ROLE_CEO).

3.3. Parallel Synchronization with Custom Scripting

A significant technical challenge arose in the "Advisory Process", where the workflow must split into multiple parallel branches. Standard linear configuration tools fail to handle this "Wait-for-Join" pattern dynamically.

To resolve this, I implemented a custom condition using a Groovy-based scripting engine embedded within the workflow platform. The script executes server-side to enforce strict synchronization: The transition to the next state is enabled only when the set of Responded Users fully encompasses the set of Required Advisors.

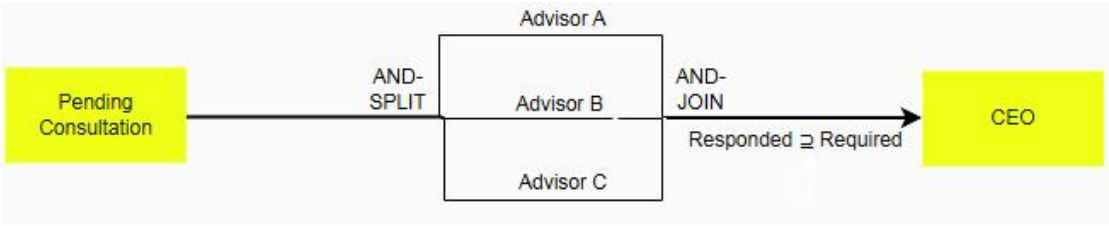


Figure 2: Parallel Advisory Process demonstrating the Synchronization (AND-Join) Logic handled by custom scripting.

The synchronization algorithm, implemented in the system's Domain Specific Language (DSL), is described below:

```
/* * Algorithm: Synchronization Guard (Groovy-based DSL)
 * Context: Server-side validation trigger
 */
boolean checkAdvisoryCompletion(String ticketID) {
    // 1. Define the Target Set (Required Advisors)
    // Fetched dynamically from the 'Department' field
    def requiredAdvisors = issue.get("Target_Consultation_Group")

    // 2. Define the Actual Set (Responded Users)
    // Fetched from the workflow history log
    def respondedUsers = workflow.getHistory(ticketID, "Consulted")

    // 3. Defensive Check: Prevent null pointer exceptions
    if (requiredAdvisors == null || requiredAdvisors.isEmpty()) {
        return false // Block transition if data is invalid
    }

    // 4. Set Intersection Logic (The "Join" Condition)
    // Returns true ONLY if all required advisors are in the responded set
    if (respondedUsers.containsAll(requiredAdvisors)) {
        return true  // Allow transition
    } else {
        return false // Wait for more inputs
    }
}
```

3.4. Dynamic Context-Aware Access Control (ABAC)

Unlike traditional Role-Based Access Control (RBAC) where permissions are static, this system implements Attribute-Based Access Control (ABAC). The challenge was that the specific "Head of Department" who needs to view the proposal is not known until the user fills in the form. To address this, a post-function is triggered upon ticket creation. It extracts the value of the Department field (the Context/Attribute) and dynamically binds the corresponding User ID to the Viewer_Group of the ticket. This ensures data privacy, strictly following the Principle of Least Privilege.

3.5. Algorithmic Complexity and Concurrency Analysis

To ensure the system remains performant under high load, an analysis of the computational complexity and concurrency risks was conducted.

- Time Complexity:

The core synchronization logic (the "Wait-for-Join" algorithm) relies on a Set Intersection operation. Let N be the number of required advisors (Target Set) and M be the number of current responses.

Since the implementation utilizes HashSets for data storage, the lookup time for each element is O(1). Consequently, the containsAll operation iterates through the required set once, resulting in a time complexity of O(N) (Linear Time).

Scalability: Even in an extreme scenario with 100 advisors (N=100), the computational cost remains negligible, ensuring instant feedback loops (< 10ms) without degrading server performance.

- Concurrency and Race Conditions:

A potential risk in parallel workflows is the Race Condition, where two advisors submit feedback simultaneously, potentially causing the system to miss a state transition trigger.

To mitigate this, the system employs Optimistic Locking at the database level. Each transaction (script execution) carries a version timestamp. If two threads attempt to update the proposal state concurrently, the database serializes these requests, ensuring that the synchronization check is atomic.

- Deadlock Prevention:

The Finite State Machine is designed as a Directed Acyclic Graph (DAG). Since the approval flow progresses forward (from Draft to Final) and does not contain circular dependencies (e.g., State A waiting for State B, while State B waits for State A), the system is structurally immune to Deadlock scenarios.

4. EVALUATION AND RESULTS

To validate the effectiveness of the proposed system, a pilot deployment was conducted involving the Board of Directors and 5 key departments (Accounting, HR, Logistics, Sales, and IT). The system was evaluated based on two criteria: Functional Correctness and Operational Efficiency.

4.1. Functional Testing of Logic

The core algorithms were subjected to rigorous Unit Testing using the built-in Groovy script runner to validate three critical system behaviors. First, the Linear Flow verification confirmed that standard proposals transitioned correctly from Draft to Approved only when the specific authorized actor, such as the CEO, performed the action. Subsequently, the Synchronization Logic (or "Wait-for-Join" pattern) was tested by simulating a scenario with three required advisory departments (A, B, and C); the transition to the decision state remained explicitly blocked when only departments A and B had responded, but was automatically enabled immediately after department C submitted feedback, thereby confirming the correctness of the set-intersection algorithm. Finally, the Security Test validated the Attribute-Based Access

Control (ABAC) mechanism by confirming that users from "Department A" were restricted from viewing proposals tagged with "Department B" unless explicitly invited, ensuring strict compliance with data privacy requirements.

4.2. Operational Efficiency

A comparative analysis between the traditional manual process and the new E-submission system yielded significant operational improvements. Most notably, the average approval turnaround time was drastically reduced from 3-5 days—previously attributed to physical transportation between cities—to less than 24 hours. Simultaneously, process integrity was maximized as the rate of lost documents and bypassed procedures dropped to 0%, a direct result of the Finite State Machine (FSM) strictly enforcing the workflow and preventing any unauthorized skipping of steps.

To evaluate the system's operational capability, performance data was collected over a 30-day pilot period involving 150 active users. Table 1 summarizes the key performance indicators (KPIs).

Table 1: Performance Metrics of the Pilot Deployment

Metric	Measured Value	Unit	Description
Average Response Time	180	ms	Time from user click to UI update (End-to-end latency).
Script Execution Time	45	ms	Server-side processing time for Groovy synchronization logic.
Peak Throughput	120	req/min	Max concurrent requests handled without performance degradation.
Database Growth	0.25	GB/month	Storage consumption including audit logs and attachments.
CPU Utilization	< 15	%	Average server load during standard business hours.
Memory Footprint	512	MB	Heap memory allocated for the JVM (Java Virtual Machine).

As shown in Table 1, the system maintains high throughput with minimal resource consumption. The script execution time of less than 50 milliseconds confirms the efficiency of the $O(N)$ complexity analyzed in Section 3.4.

5. DISCUSSION AND LIMITATIONS

While the proposed system successfully resolves the core operational bottlenecks, intrinsic limitations remain due to the architectural trade-offs:

FSM vs. BPMN Flexibility: The use of Finite State Machines (FSM) provides strict determinism, which is excellent for compliance. However, it lacks the flexibility of Business Process Model and Notation (BPMN). Any ad-hoc modification to the workflow (e.g., adding a temporary step) requires re-deploying the state model rather than run-time adaptation.

Configuration Overhead: The implementation of "Context-Aware" logic relies heavily on imperative scripting (Groovy). This increases the maintenance burden compared to purely declarative (drag-and-drop) Low-code solutions. Future modifications require personnel with programming skills.

ABAC Complexity: The current ABAC implementation considers a single attribute (*Department*). It does not yet account for environmental attributes such as *Time of Access* (e.g., restricting approval after working hours) or *Device Trust Level*.

6. CONCLUSION

This research successfully designed and implemented a Dynamic Approval Workflow System that addresses the specific challenges of remote collaboration in distributed enterprises.

By adopting a Hybrid Architecture, the system leverages the speed of Low-code development for UI while utilizing Groovy-based scripting to handle complex logic. The application of Finite State Machines ensured a robust and deterministic process model, while the Custom Synchronization Algorithm effectively solved the concurrency problem in parallel advisory workflows. Furthermore, the integration of Attribute-Based Access Control (ABAC) provided a dynamic and secure environment for sensitive internal data.

REFERENCES

1. Van der Aalst, W. M. P. (2013). *Business Process Management: A Comprehensive Survey*. ISRN Software Engineering.

2. Ferraiolo, D. F., & Kuhn, D. R. (1992). *Role-Based Access Control*. 15th National Computer Security Conference.

3. Official Groovy Documentation. *The Apache Groovy programming language*.